RAG System Design & Architecture Reference

A production guide to 10-layer retrieval-augmented generation — from document ingest through hybrid retrieval, ANN reranking, confidence scoring, constrained generation, hallucination fallback, continuous evals, caching, and full-stack observability.

Version	1.0
Scope	Production RAG pipeline — design + architecture
Layers	10 (ingest → observability)
Format	Reference + decision tables + tech stack

01 Pipeline overview

The RAG pipeline is organized as ten sequential layers. Each layer has a single responsibility, a defined interface, and observable outputs. Layers are independently deployable services connected through an event bus for async feedback and cache invalidation.

01 Ingest + normalize	Parse, OCR, dedup, chunk with overlap, tag metadata, language detect
02 Hybrid retrieval	BM25 sparse + dense embeddings in parallel, RRF score fusion
03 ANN + two-stage rerank	HNSW approximate search → cross-encoder → LLM reranker
04 Confidence scoring	Semantic sim + freshness decay + authority → composite [0,1] score
05 Constrained generation	Grounding prompt, low temperature, output schema enforcement
06 Citation-backed resp.	Span-level attribution, inline [N] anchors, source cards
07 Hallucination fallback	NLI entailment check → abstain or re-retrieve path
08 Continuous evals	P@k, ROUGE, BERTScore, RAGAS faithfulness, CI golden set
09 Caching + memory	Semantic cache, KV result cache, session window, user profile
10 Observability	OTel spans on every stage, Prometheus metrics, Grafana dashboards

02 Layer-by-layer detail

01 Ingest + normalize

- Connectors: PDF/DOCX (pdfplumber), HTML (trafilatura), DB (SQLAlchemy), REST APIs, plain text
- OCR with Tesseract or AWS Textract for scanned documents
- Deduplication via MinHash LSH on chunk fingerprints
- Chunking strategy: fixed-size with 15% overlap, or semantic sentence-boundary splitting
- Metadata tagging: source URI, ingestion timestamp, doc type, language, section heading
- Publish ingest events to message bus for downstream index updates

02 Hybrid retrieval

- BM25 via Elasticsearch or OpenSearch; inverted index on normalized tokens
- Dense embeddings via OpenAI text-embedding-3-large, Cohere embed-v3, or local BGE-M3
- Both retrieval paths run in parallel; timeout at 80ms for ANN, 50ms for BM25
- Query rewriting: HyDE (hypothetical document embeddings) or multi-query expansion
- RRF (Reciprocal Rank Fusion) merges ranked lists: score = sum(1 / (k + rank_i))
- Dedup merged list by chunk_id before passing to reranker

03 ANN + two-stage reranking

- HNSW index (Qdrant, Weaviate, or pgvector) with ef_search tuned for recall vs latency
- Stage 1: cross-encoder reranker (BGE-reranker-large) narrows k=100 → 20 candidates
- Stage 2: LLM reranker (GPT-4o-mini with listwise prompt) narrows 20 → top-10
- Latency gate: cross-encoder < 200ms, LLM reranker < 500ms; skip stage 2 under load
- Candidate pool size k tunable per query type; factual = 50, conversational = 20

04 Source confidence scoring

- Semantic similarity: cosine(query_embedding, chunk_embedding) — primary signal
- Freshness: time-decay weight = exp(-lambda * days_since_updated); lambda=0.01 default
- Authority: per-domain weight assigned at ingest time (e.g. internal wiki = 1.0, forum = 0.6)
- Composite score = 0.6 * sim + 0.2 * freshness + 0.2 * authority
- Threshold gate: drop chunks with composite score < tau (default 0.45)
- Low-confidence path: if fewer than 3 chunks pass gate, expand query and re-retrieve

05 Constrained generation

- System prompt: 'Answer ONLY using the context below. If unsure, say so.'
- Context window packed in descending confidence order; trim to fit token budget
- Temperature 0.1–0.3 for factual queries; top_p 0.9; presence_penalty 0.1
- Structured output schema (JSON mode) for downstream citation assembly
- Stop sequences prevent generation beyond defined answer boundary
- Retry budget: up to 2 regenerations before routing to abstain path

06 Citation-backed responses

- Span attribution: model instructed to emit [SOURCE_N] anchors inline
- Post-process: map each [SOURCE_N] to the chunk_id from retrieval context
- Citation card fields: document title, page/section, URL, ingestion date, confidence score
- Verification pass: confirm each cited chunk actually supports the attributed claim
- Uncited claims flagged with confidence = 0; surfaced in response metadata

07 Hallucination fallback

- NLI check: cross-encoder scores entailment between response and each cited chunk
- Faithfulness score = mean entailment across all cited spans
- Score < 0.7 → retry generation with stricter grounding prompt
- Score < 0.5 after retry → abstain: return 'I don't have reliable information on this'
- Self-consistency check: 3 independent samples voted at score < 0.6
- All fallback events logged with full trace for eval dataset collection

08 Continuous evals

- Retrieval metrics: Precision@k, Recall@k, MRR — run nightly on golden question set
- Generation metrics: ROUGE-L, BERTScore, answer correctness vs reference
- Faithfulness: RAGAS faithfulness score, TruLens groundedness
- CI gate: block deploy if any metric regresses > 5% from rolling 7-day baseline
- Eval store: golden set of 200+ queries with expected answers and source spans
- Human-in-the-loop labeling pipeline for ongoing golden set expansion

09 Caching + memory

- Semantic query cache: store query embedding + top-n result set; cosine hit threshold 0.92
- KV result cache: Redis with TTL 15min; invalidated on source document update events
- Cache hit rate target > 40%; below 35% indicates golden query set needs expansion
- Session memory: last 10 turns in Redis; used for query contextualization
- User profile store: Postgres — topic preferences, feedback history, access permissions
- Long-term entity KB: named entities extracted at ingest, stored in graph (Neo4j)

10 Observability

- Distributed tracing: OpenTelemetry spans on every service boundary, single correlation_id
- Metrics: retrieval latency histogram, cache hit rate, faithfulness score distribution
- Logs: structured JSON — query, retrieved chunk IDs, scores, model call duration
- Alerting: PagerDuty on P99 > 5s, faithfulness < 0.6 over 5min window, error rate > 1%
- Dashboards: Grafana — request volume, latency percentiles, eval metric trends
- Feedback loop: thumbs up/down + corrections written to eval store for retraining

03 System design

Three-plane architecture

The system is organized into three horizontal planes with cross-cutting infra:

Client plane	API gateway, auth + rate limiting, semantic cache. Cache hits return in <20ms and never touch the serving plane.
Serving plane	Six microservices: query service, retrieval service, generation service, confidence service, citation service, hallucination guard. Connected by async event bus.
Data plane	Vector store (HNSW), keyword index (BM25), doc store (S3 + Postgres), KG layer (Neo4j), cache layer (Redis), memory store, eval store.
Cross-cutting infra	Observability (OTel), CI/evals, config/secrets (Vault), feature flags, model registry, data lineage, RBAC, backup/DR, PII compliance.

Request lifecycle — critical path

#	Step	What happens
1	**Query in**	Normalize, detect language, extract intent
2	**Cache lookup**	Semantic cosine check; hit → skip to step 9
3	**Query rewrite**	HyDE / expansion / multi-query
4	**Parallel retrieval**	BM25 + ANN fan-out, merge with RRF
5	**Two-stage rerank**	Cross-encoder then LLM reranker
6	**Confidence gate**	Score + filter; low-conf → expand and retry step 4
7	**Constrained gen.**	Grounding prompt + low-temp LLM call
8	**Hallucination check**	NLI entailment; fail → retry or abstain
9	**Citation assembly**	Span attribution + source card construction
10	**Response out**	Structured payload with citations and confidence

04 Technology stack

Vector store	`Qdrant, Weaviate, pgvector, Pinecone`	HNSW index; Qdrant recommended for self-hosted
Keyword index	`Elasticsearch, OpenSearch`	BM25 + inverted index; also handles metadata filters
Doc store	`S3 + Postgres`	Raw files in S3, chunk metadata in Postgres
Embeddings	`text-embedding-3-large, BGE-M3, Cohere v3`	BGE-M3 for multilingual or air-gapped deployments
Reranker	`BGE-reranker-large, Cohere rerank-v3`	Cross-encoder; Cohere for SaaS, BGE for self-hosted
LLM	`GPT-4o, Claude 3.5 Sonnet, Llama-3-70B`	Lower temp (0.1-0.3) for factual RAG tasks
Cache	`Redis 7 (cluster mode)`	Semantic cache + KV result cache + session store
Graph DB	`Neo4j, Amazon Neptune`	Entity + relation KG layer; optional but improves recall
Message bus	`Apache Kafka, AWS SQS`	Async ingest events, cache invalidation, eval triggers
Tracing	`OpenTelemetry + Jaeger / Tempo`	Span every service boundary; single correlation_id
Metrics	`Prometheus + Grafana`	Latency histograms, faithfulness score, cache hit rate
Eval framework	`RAGAS, TruLens, DeepEval`	RAGAS faithfulness + context precision are the key metrics
Orchestration	`Kubernetes + Helm`	Horizontal pod autoscaling on retrieval + generation pods
Secrets / config	`HashiCorp Vault, AWS Secrets Manager`	Never embed API keys in container images
Ingest parsing	`pdfplumber, trafilatura, Docling`	Docling for complex PDF layouts with tables/figures

05 Latency budget

Target SLAs per stage. All times are wall-clock from service entry to exit. Instrument with OTel spans and alert on P99 breaches.

Cache lookup	**<5 ms**	<15 ms	Check Redis cluster health
BM25 retrieval	**<30 ms**	<80 ms	Reduce index size / add shards
ANN retrieval	**<50 ms**	<120 ms	Tune HNSW ef_search parameter
Cross-enc rerank	**<120 ms**	<300 ms	Reduce candidate pool (k)
LLM rerank	**<200 ms**	<500 ms	Cache top-n, batch requests
Generation	**<800 ms**	<2000 ms	Stream tokens, reduce context
NLI check	**<100 ms**	<250 ms	Lightweight model or skip for low-risk
Total (cache miss)	**<1.5 s**	<3.5 s	Profile with OTel trace
Total (cache hit)	**<20 ms**	<60 ms	Verify Redis hit rate > 40%

Scaling levers

- Cache hit rate is the most powerful lever. Every 10% increase in hit rate reduces average latency by ~150ms and LLM cost by ~12%. Target >40% semantic cache hit rate within the first month of production traffic.
- The two-stage reranker is the second biggest latency cost. Under high load, skip the LLM reranker (stage 2) and serve from cross-encoder output only. A/B test shows <4% quality degradation at >2x throughput.
- Generation latency is dominated by output token count. Cap max_tokens to 512 for most RAG use cases. Stream tokens to the client to reduce perceived latency.
- Vector search scales horizontally — add replicas. BM25 index sharding is the harder problem; shard by document creation date to keep hot shards small.

06 Key design decisions

Separate vector and keyword indexes?	Yes. Different scaling profiles and failure modes. Dual-path retrieval improves recall ~18% over vector-only.
When to skip LLM reranker?	Under P99 > 400ms load or when query confidence is high (>0.85). Feature flag to disable per request type.
Confidence threshold tau value?	Start at 0.45. Calibrate against human-labeled relevance on your domain corpus. Raise to 0.55 for high-stakes queries.
Chunk size?	256-512 tokens with 10-15% overlap is a good default. Smaller chunks improve precision; larger improve recall for multi-sentence answers.
When to abstain vs re-retrieve?	If faithfulness < 0.5 after one retry: abstain. If 0.5-0.7 and query is answerable: expand query and re-retrieve once more.
Embedding model choice?	Same model at ingest and query time — non-negotiable. Re-index all documents when changing embedding models.
Session memory depth?	Last 10 turns is sufficient for most conversational RAG. Beyond 10, compress with a summarization step to stay within context budget.
PII in documents?	Detect and redact at ingest time (Presidio). Log chunk IDs not raw text. Apply RBAC at chunk retrieval so users only see permitted sources.
Single LLM or multiple?	Use a cheap model for reranking and NLI checks (GPT-4o-mini, Claude Haiku), expensive model only for final generation.
How to handle multi-modal docs?	Extract images at ingest, caption with vision model, embed captions alongside text chunks. Store raw images in S3.

07 Evaluation framework

Metrics hierarchy

Metric	Tool	Target	What it measures
Faithfulness	`RAGAS`	**>0.80**	Are claims supported by retrieved context?
Context precision	`RAGAS`	**>0.75**	Is retrieved context relevant to the question?
Context recall	`RAGAS`	**>0.70**	Did retrieval find all necessary information?
Answer correctness	`RAGAS`	**>0.75**	Does answer match ground truth?
Groundedness	`TruLens`	**>0.80**	Alternative faithfulness signal
P@10	`Custom`	**>0.60**	Retrieval precision at 10 candidates
MRR	`Custom`	**>0.70**	Mean reciprocal rank of first relevant result
Latency P99	`OTel`	**<3.5s**	End-to-end wall-clock time at 99th percentile
Cache hit rate	`Redis`	**>40%**	Fraction of queries served from semantic cache
Abstain rate	`Custom`	**<5%**	Fraction of queries answered with 'I don't know'

CI eval pipeline

- On every PR: run unit tests on retrieval logic + generation prompts
- On merge to main: run golden set evaluation (200+ queries) — block deploy if metrics regress >5%
- Nightly: run full eval suite + latency benchmarks against production traffic sample
- Weekly: human review of 50 randomly sampled responses + abstain events
- Monthly: golden set expansion from human-labeled production queries

08 Operational runbook

Common failure modes and mitigations

High abstain rate (>5%)

Check confidence threshold — may be too high. Inspect top queries that abstain. Likely gaps in document corpus. Trigger targeted ingest for missing topics.

Low faithfulness score

Usually a grounding prompt regression. Check recent system prompt changes. Verify context packing is ordering chunks by confidence descending.

High retrieval latency

Check HNSW ef_search — may have been increased after index rebuild. Verify Redis cluster health. Check for index fragmentation in Elasticsearch.

Stale citations

Cache TTL may be too long for fast-changing source documents. Verify event bus is publishing doc-update events and triggering cache invalidation.

Embedding drift

Happens when embedding model is updated without re-indexing. Never partial-re-index — always re-embed the full corpus atomically.

Context window overflow

Reduce chunk size or trim context more aggressively. Check for chunks with very high token counts (tables, code blocks) — truncate at ingest.

High error rate on LLM calls

Check for prompt injection in user queries. Verify API key rotation. Implement exponential backoff with jitter. Route to fallback model if primary unavailable.

Deployment checklist

- ■ Golden set eval passes all metric thresholds in CI
- ■ OTel instrumentation verified on all new service boundaries
- ■ Feature flags set — LLM reranker can be disabled without redeploy
- ■ Redis cache warmed with top-1000 queries before traffic shift
- ■ Rollback plan documented: previous embedding model version pinned
- ■ Alert thresholds reviewed and PagerDuty escalation path confirmed
- ■ RBAC policies audited — no over-permissioned chunk access
- ■ PII redaction pipeline tested on sample of new document types